Topic5:Enumerations,StructuresandClasses

Task1:LibraryItemModel

```
enumBookStatus{
caseavailable
caseborrowed(by:String,dueIns:Int)
casereserved(for:String)

varlabel:String{
switchself{
case.available:
return"Available"
case.borrowed(letname,letdays):
return"Borrowedby\(name),duein\(days)days"
case.reserved(letname):
return"Reservedfor\(name)"
}
}
}

structBook{
vartitle:String
varstatus:BookStatus=.available
}

classMember{
varname:String
varborrowedBooks:[String]

init(name:String,borrowedBooks:[String]=[]){
self.name=name
self.borrowedBooks=borrowedBooks
}
}

varbook1=Book(title:"SwiftProgramming")
```

```
varbook2=book1
book2.status=.borrowed(by:"Gayu",dueIns:5)

print("Book1:",book1.title,"-",book1.status.label)
print("Book2:",book2.title,"-",book2.status.label)

letmember1=Member(name:"Gayu")
letmember2=member1
member2.name="Priya"

print("Member1:",member1.name)
print("Member2:",member2.name)
```

output:

Output

```
Book 1: Swift Programming - Available
Book 2: Swift Programming - Borrowed by Gayu, due in 5 days
Member 1: Priya
Member 2: Priya
```

Task2:ShapesandAreas

```
importFoundation

enumShape{
casecircle(radius:Double)
caserectangle(width:Double,height:Double)
casetriangle(base:Double,height:Double)

vararea:Double{
switchself{
case.circle(letradius):
returnDouble.pi*radius*radius
case.rectangle(letwidth,letheight):
```

```
returnwidth*height
case.triangle(letbase,letheight):
return0.5*base*height
}
}
}

letcircle=Shape.circle(radius:5)
letrectangleShape=Shape.rectangle(width:4,height:6)
lettriangle=Shape.triangle(base:8,height:5)

print("Circlearea:",circle.area)
print("Rectanglearea:",rectangleShape.area)
print("Trianglearea:",triangle.area)

structRectangle{
varwidth:Double
varheight:Double

varperimeter:Double{
return2*(width+height)
}

mutatingfuncscale(byfactor:Double){
width*=factor
height*=factor
}
}

varrect1=Rectangle(width:4,height:3)
varrect2=rect1

rect2.scale(by:2)

print("Originalwidth:",rect1.width)
```

print("Originalheight:",rect1.height)

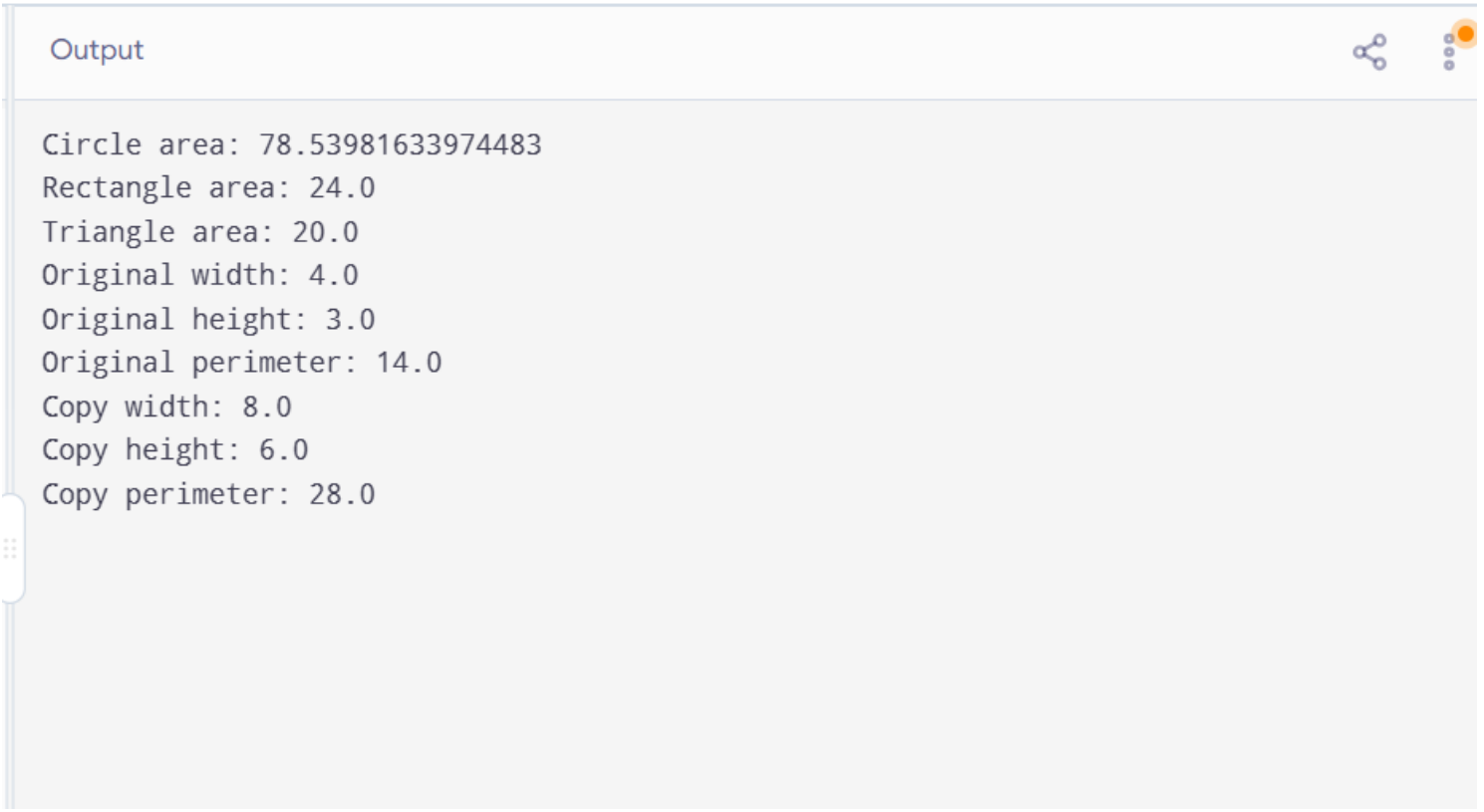
print("Originalperimeter:",rect1.perimeter)

print("Copywidth:",rect2.width)

print("Copyheight:",rect2.height)

print("Copyperimeter:",rect2.perimeter)

output:



Task3:WhyDoesThisNotCompile?

structCounter{

varvalue=0

mutatingfuncincrement(){

value+=1

}

}

classBox{

varvalue=0

funcincrement(){

```
value+=1
}
}

varc=Counter()
c.increment()
print("Countervalue:",c.value)

letb=Box()
b.increment()
print("Boxvalue:",b.value)

letb1=b
letb2=b1

print("Sameobject:",b1===b2)
```

Output:

